

Ứng dụng thư viện pb_ds của C++ trong OI

Yu Jiping

Trao đổi trại WC2015, tháng 2 năm 2015

Nguồn gốc: Ứng dụng thư viện pb_ds của C++ trong OI

Others / C++ pb_ds library in OI - Yu Jiping

Abstract

Bản gốc là một bộ slide giới thiệu pb_ds cho thí sinh OI. Tài liệu trích xuất từ PDF không giữ được mã Unicode của chữ Hán, nhưng các tên giao diện, đoạn mã và bảng số liệu vẫn đọc được. Bản dịch này chuyển nội dung thành một bài viết tự chứa bằng tiếng Việt, giữ thứ tự trình bày và các dữ liệu kỹ thuật chính của slide: hàng đợi ưu tiên, cây cân bằng, cách tự định nghĩa Node_Update, bảng băm và các so sánh hiệu năng.

Contents

1	Tổng quan	2
2	priority_queue	2
2.1	Khai báo và giao diện cơ bản	2
2.2	Các thao tác mở rộng	3
2.3	Độ phức tạp	3
2.4	So sánh thực nghiệm	4
2.4.1	Chỉ chèn rồi lấy ra	4
2.4.2	Hợp nhất heap	4
2.4.3	Dijkstra với sửa khóa	4
3	tree	5
3.1	Khai báo và ý nghĩa tham số	5
3.2	Ghi chú về splay tree	6
3.3	Thống kê thứ tự	6
3.4	Ghép và tách cây	7
4	Tự định nghĩa Node_Update	7
4.1	Khung cơ bản	7
4.2	Tính tổng tiền tố và tổng đoạn	7
4.3	Benchmark cho tree	8

5	hash_table	9
5.1	Khai báo	9
5.2	Khi nào dùng	9
6	Tổng kết	9

1 Tổng quan

pb_ds là viết tắt của **Policy-Based Data Structures**. Đây là một phần mở rộng của GNU libstdc++, cung cấp các cấu trúc dữ liệu có chính sách cấu hình được. Tài liệu chính thức mô tả mục tiêu của thư viện là đạt hiệu năng cao, linh hoạt, an toàn về mặt ngữ nghĩa và tương thích với các container tương ứng trong std và std::tr1.

Các cấu trúc thường dùng trong OI gồm:

- priority_queue: nhiều loại heap, có thể sửa hoặc xóa phần tử thông qua iterator, và có thể hợp nhất heap;
- tree: một container kiểu set/map, có thể dùng cây đồ-dân, splay hoặc ordered-vector tree;
- hash_table: bảng băm dạng chaining hoặc probing;
- ngoài phạm vi chính của bài còn có trie và một số cấu trúc khác.

So với STL, pb_ds không phải là một thay thế tuyệt đối. Điểm mạnh của nó là cung cấp sẵn những thao tác vốn thường phải tự viết trong bài thi: sửa khóa trong heap, hợp nhất nhiều heap, hỏi hạng trong cây cân bằng, tách và ghép cây theo giá trị khóa, hoặc dùng bảng băm nhanh hơn std::map trong những bài chỉ cần ánh xạ khóa sang giá trị. Tài liệu cũng nhắc rằng khi dùng trong các kỳ thi, cần kiểm tra trước môi trường biên dịch và quy định của kỳ thi, vì đây là phần mở rộng của GNU.

2 priority_queue

2.1 Khai báo và giao diện cơ bản

Hàng đợi ưu tiên của pb_ds nằm trong header sau:

```
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
```

Cách dùng cơ bản giống std::priority_queue: có size(), empty(), push(const T), top(), pop() và clear(). Mẫu lớp trong slide được ghi như sau:

```
template <typename Value_Type,
          typename Cmp_Fn = std::less<Value_Type>,
          typename Tag = pairing_heap_tag,
          typename Allocator = std::allocator<char> >
class priority_queue;
```

Trong đó:

- Value_Type là kiểu phần tử;

- Cmp_Fn là hàm so sánh, mặc định là `std::less`;
- Tag chọn loại heap bên dưới;
- Allocator thường giữ mặc định.

Các tag heap được nhắc trong slide gồm:

- `binary_heap_tag`: heap nhị phân;
- `binomial_heap_tag`: heap nhị thức;
- `rc_binomial_heap_tag`: heap nhị thức có redundant counter;
- `pairing_heap_tag`: pairing heap;
- `thin_heap_tag`: một biến thể gần với Fibonacci heap.

2.2 Các thao tác mở rộng

Ngoài những thao tác giống STL, `pb_ds::priority_queue` có thể duyệt bằng `begin()` và `end()`, sửa khóa, xóa một phần tử cụ thể và hợp nhất hai heap. Điểm quan trọng là `push` trả về `point_iterator`; iterator này dùng lại trong `modify` hoặc `erase`.

```
point_iterator push(const_reference)
void modify(point_iterator, const_reference)
void erase(point_iterator)
void join(priority_queue &other)
```

Ví dụ trong slide:

```
priority_queue<int> p;
priority_queue<int>::point_iterator it = p.push(0);
p.push(1);
p.push(2);
p.modify(it, 3);
assert(p.top() == 3);
p.erase(it);
assert(p.top() == 2);
```

Hàm `join` nhập toàn bộ phần tử của `other` vào heap hiện tại; sau thao tác này `other` trở thành rỗng. Đây là thao tác rất tiện trong các bài cần nhiều heap có thể hợp nhất, chẳng hạn những mô hình thường viết bằng leftist heap hoặc skew heap.

2.3 Độ phức tạp

Các thao tác được slide gom lại thành năm nhóm: `push`, `pop`, `modify`, `erase` và `join`. Bảng dưới diễn giải các ghi chú về độ phức tạp.

Loại heap	Độ phức tạp được nhấn mạnh
<code>pairing_heap_tag</code>	<code>push</code> và <code>join</code> là $O(1)$, các thao tác còn lại khấu hao $\Theta(\log n)$
<code>binary_heap_tag</code>	chỉ hỗ trợ <code>push</code> và <code>pop</code> ; cả hai khấu hao $\Theta(\log n)$
<code>binomial_heap_tag</code>	<code>push</code> khấu hao $O(1)$, các thao tác còn lại $\Theta(\log n)$
<code>rc_binomial_heap_tag</code>	<code>push</code> là $O(1)$, các thao tác còn lại $\Theta(\log n)$
<code>thin_heap_tag</code>	<code>push</code> là $O(1)$; không hỗ trợ <code>join</code> ; các thao tác còn lại $\Theta(\log n)$, trong đó <code>modify</code> theo hướng tăng khóa có thể đạt $O(1)$ khấu hao

Kinh nghiệm trong slide là: nếu bài cần hợp nhất heap, thường ưu tiên `pairing_heap_tag`; nếu bài giống Dijkstra và cần sửa khóa, có thể thử `thin_heap_tag`, nhưng kết quả thực tế còn phụ thuộc dữ liệu và hằng số.

2.4 So sánh thực nghiệm

PDF chứa nhiều bảng benchmark. Các bảng này không phải là kết luận phổ quát, nhưng cho thấy hằng số trong thực tế quan trọng không kém độ phức tạp tiệm cận.

2.4.1 Chỉ chèn rồi lấy ra

Thử nghiệm đầu tiên sinh 10^7 khóa ngẫu nhiên trong $[0, 2^{30})$, chèn vào heap rồi lần lượt pop. Bảng có hai cột thời gian theo mili-giây.

Cấu trúc	Cột 1	Cột 2
<code>std::priority_queue</code>	16378 ms	2797 ms
<code>binary_heap_tag</code>	5454 ms	2720 ms
<code>pairing_heap_tag</code>	21421 ms	17362 ms
<code>thin_heap_tag</code>	39290 ms	28672 ms
heap tự viết	8692 ms	4916 ms
<code>push_back + sort + pop_back</code>	4563 ms	938 ms

Điểm đáng chú ý là với một lô dữ liệu đã biết trước, sắp xếp mảng rồi lấy dần từ cuối có thể nhanh hơn mọi heap. Điều này chỉ dùng được khi toàn bộ dữ liệu đã có sẵn và không cần xen kẽ thao tác động.

2.4.2 Hợp nhất heap

Thử nghiệm kế tiếp sinh 4194304 khóa trong $[0, 2^{30})$, phân vào nhiều heap rồi hợp nhất. Heap nhị phân của `pb_ds` không có join thật sự nên cách hợp nhất tương ứng phải chèn lại phần tử.

Cấu trúc	Cột 1	Cột 2
<code>std::priority_queue + hợp nhất thủ công</code>	38746 ms	4642 ms
<code>binary_heap_tag</code>	5605 ms	3127 ms
<code>binary_heap_tag + hợp nhất thủ công</code>	11206 ms	3955 ms
<code>pairing_heap_tag</code>	6954 ms	5328 ms
<code>thin_heap_tag</code>	14514 ms	9588 ms

2.4.3 Dijkstra với sửa khóa

Với đồ thị có 10^6 đỉnh và $2 \cdot 10^7$ cạnh, slide so sánh các heap trong một cài đặt Dijkstra có cập nhật khoảng cách. `binary_heap_tag` không hỗ trợ sửa khóa nên không xuất hiện trong bảng.

Cấu trúc	Cột 1	Cột 2
<code>binomial_heap_tag</code>	26807 ms	17018 ms
<code>pairing_heap_tag</code>	7684 ms	6230 ms
<code>rc_binomial_heap_tag</code>	32004 ms	18745 ms
<code>thin_heap_tag</code>	9388 ms	6483 ms
heap tự viết	9595 ms	5080 ms

Một nhóm thử nghiệm khác cho Dijkstra dùng nhiều kích thước đồ thị khác nhau. Hai bảng dưới giữ nguyên số liệu của slide; tên một cột trong PDF là heap tự viết.

<i>n</i>	<i>m</i>	pairing	thin	Fib	heap tự viết	cấu trúc khác
100k	10M	390	578	703	1088	422
1M	10M	2063	3516	4209	9236	2594
1M	20M	4298	6595	6859	18648	4745
100k	25M	3375	4805	3735	5655	2501
1M	30M	6579	9609	9393	27960	6816
50k	50M	484	563	1235	6392	453
100k	50M	7061	9550	8051	11301	4580
1M	50M	11088	15559	14707	47108	10848

<i>n</i>	<i>m</i>	pairing	thin	Fib	heap tự viết	cấu trúc khác
100k	10M	281	360	438	625	219
1M	10M	1485	2250	2836	7968	1444
1M	20M	3204	4485	4987	16629	2711
100k	25M	2141	2798	3206	2309	1262
1M	30M	4908	6642	6795	25128	3964
50k	50M	375	407	680	2217	280
100k	50M	4245	5792	4150	4105	2312
1M	50M	8455	11541	10927	42101	6217

Kết luận của phần này là:

- `binary_heap_tag` thường chỉ hơn `std::priority_queue` trong một số trường hợp đơn giản, nhưng không có các thao tác mở rộng;
- `pairing_heap_tag` thường thực dụng hơn `binomial_heap_tag` và `rc_binomial_heap_tag`;
- nếu chỉ có `push`, `pop` và `join`, cần cân nhắc dữ liệu cụ thể; đôi khi heap nhị phân vẫn nhanh;
- nếu cần `modify`, nên thử `pairing_heap_tag` và `thin_heap_tag`, sau đó đo lại trên bài cụ thể.

3 tree

3.1 Khai báo và ý nghĩa tham số

Container `tree` nằm trong hai header:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
```

Slide viết dạng đơn giản là `tree<Key, T>`. Nếu `T` là một kiểu dữ liệu thật, container hoạt động như `map`; nếu muốn kiểu `set`, dùng `null_type` cho tham số `mapped`. Một số môi trường cũ từng dùng `null_mapped_type`; khi biên dịch thực tế cần theo đúng phiên bản thư viện.

Mẫu lớp đầy đủ:

```
template <
    typename Key,
    typename Mapped,
    typename Cmp_Fn = std::less<Key>,
    typename Tag = rb_tree_tag,
    template <
        typename Const_Node_Iterator,
```

```

    typename Node_Iterator,
    typename Cmp_Fn_,
    typename Allocator_>
class Node_Update = null_tree_node_update,
    typename Allocator = std::allocator<char> >
class tree;

```

Trong đó:

- Key, Mapped, Cmp_Fn tương tự std::map;
- Tag chọn cấu trúc bên dưới: rb_tree_tag, splay_tree_tag hoặc avl_tree_tag;
- Node_Update cho phép mỗi nút lưu thêm metadata và cập nhật metadata đó khi cây xoay hoặc thay đổi.

Vì tree là associative container, nó có nhiều thao tác giống std::map: begin(), end(), size(), empty(), clear(), find, lower_bound, upper_bound, erase, insert và operator[].

3.2 Ghi chú về splay tree

Slide có một đoạn nhắc vui rằng nếu không tự viết splay trong bài, có thể dùng splay của pb_ds. Mã nguồn libstdc++ từng chứa dòng chú thích dạng leetspeak trong file sau:

```

ext/pb_ds/detail/splay_tree/splay_tree.hpp
// $p14y 7r33 7481.

```

Nội dung kỹ thuật của ghi chú này là: splay_tree_tag có sẵn, nhưng hằng số không nhất thiết nhỏ hơn cài đặt tay trong mọi bài. Nếu bài chỉ cần container có thứ tự thông thường, rb_tree_tag thường là lựa chọn ổn định hơn.

3.3 Thống kê thứ tự

pb_ds cung cấp sẵn tree_order_statistics_node_update. Khi dùng policy này, cây có thêm hai hàm:

```

iterator find_by_order(size_type order)
size_type order_of_key(const_key_reference r_key)

```

find_by_order(order) trả về iterator trỏ đến phần tử nhỏ thứ order + 1; chỉ số bắt đầu từ 0. Nếu order vượt quá kích thước cây, hàm trả về end().

order_of_key(r_key) trả về số phần tử trong cây nhỏ hơn r_key. Vì vậy nó chính là hạng 0-indexed của khóa nếu khóa có trong cây, và là vị trí chèn nếu khóa chưa tồn tại.

Ví dụ mẫu cho một ordered set:

```

tree<int, null_type, std::less<int>, rb_tree_tag,
    tree_order_statistics_node_update> T;

T.insert(10);
T.insert(20);
T.insert(30);

assert(*T.find_by_order(1) == 20);
assert(T.order_of_key(25) == 2);

```

3.4 Ghép và tách cây

tree còn có hai thao tác đặc biệt:

```
void join(tree &other)
void split(const_key_reference r_key, tree &other)
```

`join(other)` hợp nhất `other` vào cây hiện tại. Hai cây phải có miền khóa rời nhau theo thứ tự: toàn bộ khóa của một cây nhỏ hơn toàn bộ khóa của cây còn lại. Nếu điều kiện này không đúng, thư viện có thể ném ngoại lệ. Sau khi ghép, `other` rỗng.

`split(r_key, other)` trước hết làm rỗng `other`, sau đó chuyển các phần tử có khóa lớn hơn `r_key` từ cây hiện tại sang `other`. Các phần tử không lớn hơn `r_key` ở lại trong cây ban đầu. Với comparator đảo chiều, ý nghĩa “lớn hơn” phải hiểu theo thứ tự do comparator định nghĩa.

4 Tự định nghĩa Node_Update

4.1 Khung cơ bản

Một `Node_Update` tự định nghĩa là một template nhận các kiểu iterator nội bộ của cây, comparator và allocator. Khung trong slide:

```
template <class Node_Citr, class Node_Itr,
          class Cmp_Fn, class _Alloc>
struct my_node_update {
    virtual Node_Citr node_begin() const = 0;
    virtual Node_Citr node_end() const = 0;
    typedef int metadata_type;
};
```

`metadata_type` là kiểu dữ liệu phụ được lưu ở mỗi nút. Trong ví dụ của slide, `metadata` là `int` dùng để lưu tổng giá trị trong cây con.

Khi cây thay đổi, thư viện gọi `operator()` của policy để cập nhật `metadata` tại một nút. Ví dụ:

```
inline void operator()(Node_Itr it, Node_Citr end_it) {
    Node_Itr l = it.get_l_child(), r = it.get_r_child();
    int left = 0, right = 0;
    if (l != end_it) left = l.get_metadata();
    if (r != end_it) right = r.get_metadata();
    const_cast<metadata_type&>(it.get_metadata())
        = left + right + (*it)->second;
}
```

Ở đây `it` là iterator nội bộ trỏ tới một nút của cây, không phải iterator thông thường của container. Biểu thức `(*it)->second` lấy mapped value của phần tử đang nằm trong nút đó. Các hàm `get_l_child()`, `get_r_child()` trả về con trái và con phải; `get_metadata()` truy cập `metadata` của một nút.

4.2 Tính tổng tiền tố và tổng đoạn

Khi mỗi nút lưu tổng giá trị trong cây con, ta có thể tìm tổng trên mọi khóa không lớn hơn x bằng cách đi trong cây như tìm kiếm nhị phân.

```

inline int prefix_sum(int x) {
    int ans = 0;
    Node_CItr it = node_begin();
    while (it != node_end()) {
        Node_CItr l = it.get_l_child(), r = it.get_r_child();
        if (Cmp_Fn()(x, (*it)->first)) {
            it = l;
        } else {
            ans += (*it)->second;
            if (l != node_end()) ans += l.get_metadata();
            it = r;
        }
    }
    return ans;
}

```

Từ đó có tổng trên đoạn khóa $[l, r]$:

```

inline int interval_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
}

```

Ví dụ sử dụng:

```

tree<int, int, std::less<int>, rb_tree_tag, my_node_update> T;
T[2] = 100;
T[3] = 1000;
T[4] = 10000;
printf("%d\n", T.interval_sum(3, 4));
printf("%d\n", T.prefix_sum(3));

```

Ví dụ này minh họa khả năng lấy một cây cân bằng có sẵn rồi gắn thêm thông tin động vào mỗi nút. Tuy vậy, slide cũng cảnh báo rằng cách này không phải lúc nào cũng tiện hơn tự viết cấu trúc dữ liệu. Một số thao tác như insert, find, erase, operator[], join và split vẫn có thể dùng, nhưng iterator nội bộ không dễ dùng, tài liệu về phần này ít, và nếu cần truy cập sâu vào cấu trúc cây thì đôi khi viết tay lại rõ ràng hơn.

4.3 Benchmark cho tree

Thử nghiệm đầu tiên sinh 10^7 khóa trong $[0, 2^{30})$, chèn vào container và sau đó duyệt hoặc xóa. ov_tree_tag trong thử nghiệm này không hoàn thành trong giới hạn thời gian.

Cấu trúc	Cột 1	Cột 2
std::set	16154 ms	11033 ms
rb_tree_tag	13336 ms	10909 ms
splay_tree_tag	23425 ms	15051 ms
splay tự viết	25157 ms	15811 ms
treap tự viết	20906 ms	16627 ms
cây tìm kiếm nhị phân tự viết	11783 ms	9398 ms
push_back + sort	4574 ms	949 ms

Thử nghiệm thứ hai dùng khóa trong $[0, 2^{30})$ và mapped value, so sánh container ánh xạ. Bảng trong slide còn đưa thêm hai bảng bấm để đối chiếu.

Cấu trúc	Cột 1	Cột 2
std::map	29160 ms	20598 ms
rb_tree_tag	24661 ms	20644 ms
splay_tree_tag	48712 ms	32100 ms
push_back + sort + lower_bound	15236 ms	5267 ms
cc_hash_table	5705 ms	3750 ms
gp_hash_table	5173 ms	1407 ms

Kết quả này cho thấy `rb_tree_tag` có thể cạnh tranh với `std::map/std::set`, nhưng nếu bài toán chỉ cần tra cứu tĩnh trên dữ liệu đã sắp xếp thì mảng cộng với `sort` và `lower_bound` vẫn rất mạnh. Nếu không cần thứ tự, bảng băm thường nhanh hơn đáng kể.

5 hash_table

5.1 Khai báo

Bảng băm của `pb_ds` nằm trong:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/hash_policy.hpp>
using namespace __gnu_pbds;
```

Hai lớp được nhắc trong slide:

```
cc_hash_table<Key, Mapped>
gp_hash_table<Key, Mapped>
```

`cc_hash_table` dùng kỹ thuật `separate chaining`, còn `gp_hash_table` dùng `probing` trong bảng. Cả hai cung cấp những thao tác quen thuộc như `find` và `operator[]`. Trong benchmark của slide, `gp_hash_table` là cấu trúc nhanh nhất ở bài ánh xạ khóa sang giá trị, nhưng cũng cần lưu ý rằng bảng băm không giữ thứ tự khóa và phụ thuộc vào chất lượng hàm băm.

5.2 Khi nào dùng

Trong các bài OI, bảng băm phù hợp khi cần ánh xạ khóa sang giá trị, truy vấn không yêu cầu thứ tự và dữ liệu không bị thiết kế để phá hàm băm. Nếu cần `lower_bound`, hạng, tiền tố hoặc tách theo khóa, phải dùng `tree` hoặc một cấu trúc có thứ tự khác.

6 Tổng kết

Các ý chính của slide có thể tóm lại như sau:

- `priority_queue` của `pb_ds` bổ sung những thao tác mà STL không có: `modify`, `erase` theo `iterator` và `join`;
- `tree` có thể dùng như `set/map`, đồng thời hỗ trợ `split` và `join`; với policy thống kê thứ tự, nó còn hỗ trợ hỏi hạng và tìm phần tử thứ `k`;
- tự định nghĩa `Node_Update` cho phép lưu metadata ở nút cây, nhưng phần này đòi hỏi hiểu rõ `iterator` nội bộ và không phải lúc nào cũng đơn giản hơn cài đặt tay;

- nếu chỉ cần ánh xạ khóa sang giá trị và không cần thứ tự, `cc_hash_table` hoặc `gp_hash_table` thường nhanh hơn cây;
- dù `pb_ds` rất tiện, không nên mặc định dùng nó để thay thế mọi `std::set`, `std::map` hoặc `std::priority_queue`. Cần kiểm tra môi trường, quy định cuộc thi, thao tác thật sự cần dùng và hằng số thực nghiệm của bài.

Những hình hoặc bố cục từng bước trong slide gốc không thể phục hồi từ văn bản trích xuất. Nội dung tương ứng đã được chuyển thành mô tả bằng lời và các bảng trong bản dịch này, không nhúng hình rời.